



Assessment Cover Sheet

Assessment	Project(Individual)		
Assessment	Uncontrolled	Individual	Not must-pass
Due Date	11 December 2025	Course Code	IT9002
Course Title	Natural Language Processing		
Internal Moderator's	Dr. Abdelhameed Fawzy		
External Examiner's	Dr.		

Instructions:

1. This cover sheet must be completed (section in blue below) and attached to your assessment before submission in hard copy/soft copy.
2. The time allowed for this assessment is XXX minutes/hours/days.
3. This assessment carries XXX marks distributed to a total of XXX questions assessing CILO X and CILO X.
4. The materials allowed for use in this assessment are XXX, XXX, and XXX.
5. The **use of generative AI tools is strictly prohibited.**
6. References consulted (if any) must be properly acknowledged and cited.
7. The assessment has a total of XXX pages

Learner ID	202508989	Date Submitted	08/01/2026
Learner Name	JAAFAR ISA MOHAMED ISA AHMED		
Programme	MSc in AI		
Programme	IT9002		
Lecturer's	Sini Raj Pulari		

By submitting this assessment for marking, I affirm that this assessment is my own work.

Do not write beyond this line. For assessor use

Learner

Assessor's Name	Sini Raj Pulari		
Marking Date		Maks Obtained	

Comments:

Amazon Product Reviews Sentiment Analysis

Contents

Task 1 – Problem Statement Formulation and definition	3
Introduction:	3
Problem Definition:.....	3
Motivation:	3
Problem Statement:	3
Expected Result:	3
Task 2 - Selection of an Appropriate Data Set (Data Collection)	4
Source of Dataset and Justification	4
Dataset: Amazon Product Reviews (CSV)	4
Visualize Initial Data and Shape	4
Labels Analysis and Encoding.....	5
Task 3 –Text Preprocessing.....	5
Data Cleaning	6
Tokenization	6
Stop-words Removal.....	6
Lemmatization.....	6
Text Representation	6
Visualization	6
Task 4- Text Representation	7
POS Tagging	8
Named Entity Recognition (NER)	8
Word Embeddings	8
Task 5 –Text Classification / Prediction	8
Multinomial Naive Bayes (MNB)	8
Long Short-Term Memory (LSTM)	9
Task 6 – Evaluation, Inferences, Recommendation and Reflection	10
Evaluation	10
Inferences.....	10

Recommendation.....	11
Reflections.....	12
References	12

Amazon Product Reviews Sentiment Analysis

Task 1 – Problem Statement Formulation and definition

Introduction:

Dealing with big data is difficult, especially when it is done manually, and when it is related to emotions, it is too difficult. This is where NLP comes in, the NLP defines as a field combined between computer science, artificial intelligence and computational linguistics aim to understand and interpret human language in the way that resembles humans.(Jurafsky and H. Martin 2026).

NLP analyzes emotions and turns them into something extremely easy. The dataset of Amazon customer reviews is one example of what models can be trained to read easily.

Problem Definition:

The difficulty of classifying more than 565000 records of customer reviews manually on purpose to sentiment analysis.

Motivation:

Now, with the development of e-commerce, Amazon receive a lot of daily reviews and ratings from customers about different products. For any company or vendor, especially a big company like Amazon, it is humanly impossible to manually read and rate all these reviews to understand the level of customer satisfaction. The motivation of this project is using NLP techniques to automate the process of analyzing emotions, that's enabling companies to identify product weakness points for improving their quality and enhance their marketing strategies based on real and immediate customer feedback.

Problem Statement:

The main problem is the stars' rating is not enough to provide an explanation for customer satisfaction or pouts, also the raw text observations are unstructured data that is difficult to analyze on a large scale. The aim of this project is producing a strong classification system depend on NLP technique to classify amazon product reviews automatically to positive or negative feelings. This system will solve the challenges of customers' language processing like understanding context and complex sentence structure to produce accurate evaluation reflect customer opinion that's help managers to make data-driven decisions.

Expected Result:

Trying to access a copy of revised version of Amazon reviews ready to analyse by using advanced text preprocessing techniques like Tokenization, Lemmatization, TF-IDF methods and Stemming. To obtain accurate classification models, we will effort to implement two models by using libraries like SKLEARN, NUMPY, Pandas and Keras to apply Naive Bayes and LSTM to compare their efficiency in predicting emotions. One of the most important expected Result is produce

visualizations focuses on the most of influential words in customer satisfaction or complaints. The most visualizations are word clouds and frequency charts.

Task 2 - Selection of an Appropriate Data Set (Data Collection)

Source of Dataset and Justification

Dataset: Amazon Product Reviews (CSV)

```
#Define Dataset
df = pd.read_csv('Reviews.csv')
```

(RUMI 2021)

According to trust datasets platforms, I chose (RUMI 2021) because it rich on:

- Data count, that’s help us to arrive more effectively training.
- Trust data, that’s gives us reassurance because it’s on Kaggle platform.
- Modern data, that’s help us to read customers’ trends.

According to technical aspect, the dataset contains unstructured text and that’s important for apply NLP tasks, and that’s contain star rating (1-5) and it’s help us to classify and predicant.

Visualize Initial Data and Shape

Rows = Sample Size: 568454
(records)
Columns = Variables: 10 (features)
Types: 2 Dtypes (int64, object)

```
#Print Dataset Information, Sample Size and  
Number of variables (Shape)  
print("Dataset Information")  
print(f"Sample Size: {df.shape[0]}  
customers")  
print(f"Number of variables:  
{df.shape[1]}")  
df.info()
```

Dataset Information
Sample Size: 568454 customers
Number of variables: 10
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 568454 entries, 0 to 568453
Data columns (total 10 columns):
Column Non-Null Count Dtype
0 Id 568454 non-null int64
1 ProductId 568454 non-null object
2 UserId 568454 non-null object
3 ProfileName 568428 non-null object
4 HelpfulnessNumerator 568454 non-null int64
5 HelpfulnessDenominator 568454 non-null int64
6 Score 568454 non-null int64
7 Time 568454 non-null int64
8 Summary 568427 non-null object
9 Text 568454 non-null object
dtypes: int64(5), object(5)
memory usage: 43.4+ MB

Screenshot 1

(.shape) function display the Initial inspection as structural complexity of dataset and its consist of 568,454 rows and 10 columns, and that’s give NLP model enough samples to learn various linguistic patterns and sentiments effectively.

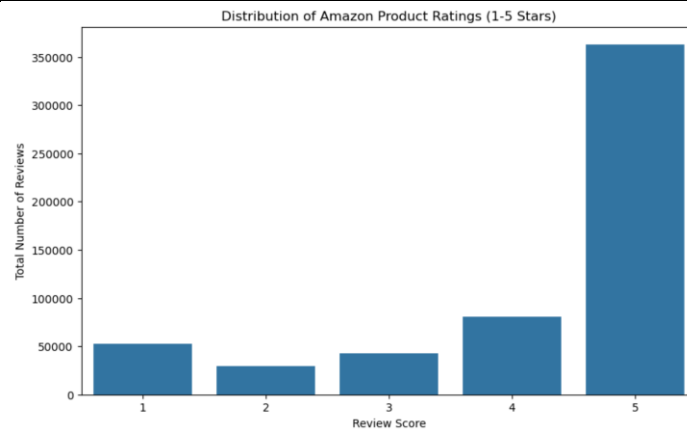
```
#Show the first 5 rows  
print("\nInitial Data Sample")  
print(df.head(5))
```

Initial Data Sample
Id ProductId UserId ProfileName
0 1 B001E4KF08 A350XH7AUH8GM delmartian
1 2 B00813GRG4 A1D87F6ZCVC9NK dll pa
2 3 B00N00C0H ABXLW6ZXXAZN Natalia Corres "Natalia Corres"
3 4 B00NUA8QIQ A3958ORC6FGVXY Karl
4 5 B006K2Z27K A1UQR5CLF8GMYT Michael D. Bigham "H. Wassir"
HelpfulnessNumerator HelpfulnessDenominator Score Time
0 1 1 5 1303862400
1 0 0 1 1346976000
2 1 1 4 1219017600
3 0 3 2 1307923200
4 0 5 1350777600
Summary Text
0 Good Quality Dog Food I have bought several of the Vitality canned d...
1 Not as Advertised Product arrived labeled as Jumbo Salted Peanut...
2 "DeLight" says it all This is a confection that has been around a fe...
3 Cough Medicine If you are looking for the secret ingredient i...
4 Great taffy Great taffy at a great price. There was a wid...

Screenshot 2

df.head(5) function display an Initial Data Sample, data contains key features such as Score (target label) and Text (raw input for NLP tasks), which will undergo significant preprocessing.

```
# Graph 1: Star Ratings
plt.figure(figsize=(10, 6))
sns.countplot(x='Score',
data=df,)
plt.title('Distribution
of Amazon Product Ratings
(1-5 Stars)')
plt.xlabel('Review
Score')
plt.ylabel('Total Number
of Reviews')
plt.show()
```



(Star rating) by column chart display positive evaluation by evaluate more than 300000 customers a 5 in rating score.

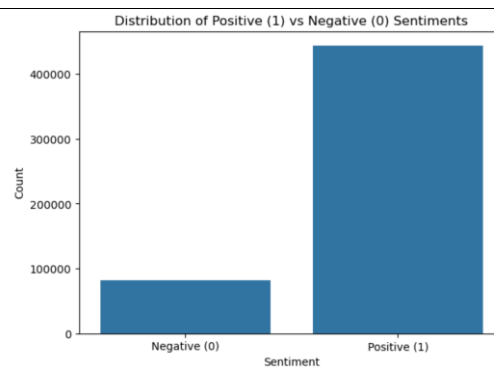
Graph 1

Labels Analysis and Encoding

For realistic evaluation we must remove 3 in Star rating because it is neutral because 3 mean no preferential data, so that confuses the outstanding model. Therefore, we used the numbers 1 and 2 as negative ratings, and the numbers 4 and 5 as positive ratings, and we define the negative rating as 0 and the positive rating as 1.

In other hand, technically reason for the exclusion is that the neutral number confuses the model and reduces accuracy, and it is better to focus on clear emotions, whether positive or negative. In addition, the algorithms such as Naive Bayes and LSTM only understand numbers and the best is use 0 and 1 for sentiment analysis.

```
# Graph 2: Star Ratings categories
plt.figure(figsize=(7, 5))
sns.countplot(x='Sentiment',
data=df_filtered)
plt.title('Distribution of Positive
(1) vs Negative (0) Sentiments')
plt.xticks([0, 1], ['Negative (0)',
'Positive (1)'])
plt.ylabel('Count')
plt.show()
```



After transformation score to categories, clear distinction between positive and negative classes, which will be used for model training.

Graph 2

Task 3 –Text Preprocessing

The primary goal is transformation the reviews from object(raw) -as unstructured text- to structural data as integers and that's appropriate to understanding of model. The review text contains noises like punctuation marks, HTML tags and stop-words that do not contribute to the sentiment and making preprocessing a critical step for achieving high model accuracy.

```
def preprocess_text(text):
    # Data Cleaning
    # Use Regex to remove any non-letter, Transform to lowercase
    text = re.sub(r'[^\w-zA-Z]', ' ', str(text).lower())

    # Tokenization
    words = text.split()

    # Stop-words Removal and Lemmatization
    lemmatizer = WordNetLemmatizer()
    stop_words = set(stopwords.words('english'))

    cleaned_words = [lemmatizer.lemmatize(w) for w in words if w not in stop_words]

    return " ".join(cleaned_words)

# Apply Data Cleaning
df_sample = df_filtered.sample(5000, random_state=42)
df_sample['Cleaned_Text'] = df_sample['Text'].apply(preprocess_text)

# Display the result for comparison
print(df_sample[['Text', 'Cleaned_Text']].head())
```

Data Cleaning

- Use Regex to remove any non-letter like numbers, #, \$, @, !, %, ^.
- Transform all text to be same by convert all letters to lowercase for help us in Tokenization.

Tokenization

Dividing a long review into single words (Tokens), and that's allowed to model to analyses each word separately.

Stop-words Removal

Remove common English word that's happens frequently and it does not express feelings like "the", "is", "and", "a" and it is reduces the amount of data and focuses on effective words, that will filtering out by using the NLTK library.

Lemmatization

Use WordNetLemmatizer to reduce words to their base dictionary form, like "running" becomes "run" and "better" becomes "good". This step is the best to stemming because it preserves the semantic meaning of the words.

Text Representation

Since model cannot read raw text, the cleaned words were converted into integers by using TF-IDF vectorizer to assigns a weight for each word based on how often it appears in a specific review face than common it is across dataset. This helps import words that define sentiment like excellent have a high value on positive and defective have a high value on negative

Visualization

For ensure of success data cleaning, two visualization was produced. Word Clouds to visual representation clarify the size of each word like "one" and "love" as high positive sentiment and "bad" as negative sentiment. Second one is comparison table to display the text pre-post cleaning.



Task 4- Text Representation

Through this task, we will apply 3 representation techniques to extract meaningful features from reviews. We will use POS Tagging, NER and Word Embeddings. And the main goal for this task is to translate human texts and words to digital vectors or arrays for the model that can perform operations on it to predict the results like classifying the review as positive or negative.

Installing collected packages: en-core-web-sm
Successfully installed en-core-web-sm-3.8.0
[+] Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')

```
--- POS Tagging ---
Amazon --> PROPN
provided --> VERB
a --> DET
great --> ADJ
phone --> NOUN
, --> PUNCT
but --> CCONJ
the --> DET
delivery --> NOUN
by --> ADP
FedEx --> PROPN
was --> AUX
late --> ADJ
. --> PUNCT

--- Named Entity Recognition (NER) ---
Entity: Amazon, Label: ORG
Entity: FedEx, Label: GPE
```

Screenshot 5

```
!python -m spacy download en_core_web_sm
# Download the English version of the model
nlp = spacy.load("en_core_web_sm")

# Applying the processes to a sample from a single
review for illustration
sample_text = "Amazon provided a great phone, but
the delivery by FedEx was late."
doc = nlp(sample_text)

# Implementing POS Tagging
print("--- POS Tagging ---")
for token in doc:
    print(f"{token.text} --> {token.pos_}")

# NER Implementation
print("\n--- Named Entity Recognition (NER) ---")
for ent in doc.ents:
    print(f"Entity: {ent.text}, Label:
{ent.label_}")
```


POS Tagging

This technique for classify each word in the customer review according to grammatical function like noun, verb, adjective and adverb. After classification, Adjectives is the key of sentiment analysis because the keywords like bad or excellent is adjective expresses directly the customers' opinion.

Named Entity Recognition (NER)

This technique for identify proper nouns and entities within the text, such as company names, people, or websites and that's helping to identify specific goal such as brands (Amazon) or delivery services mentioned in the text, allowing for a more granular analysis of the feedback. And that's help us to know the customer complain on product or shipping company (like FedEx or DHL), or perhaps a competitor, helps Amazon categorize the problem: Is it with the product quality or the logistics?

Word Embeddings

This technique for converts words into numbers (Vectors) in a way that makes words that are similar in meaning close to each other for model understanding. After that, the model will be able to understand happy and joyful have a near meaningful, and these will help LSTM model to understanding the structure of text, not just the number of words.

```
Word Embeddings (Vector representation)
Word: Amazon
Vector (First 10 values): [-1.4584436 -0.4298126  0.7456652  0.22648771  0.5593522  0.50314885
 1.5457511  0.81502676 -0.9093803 -0.02859047]
Vector Shape: (96,)
```

Screenshot 6

```
# Word Embeddings
print("\n Word Embeddings (Vector representation)")
first_token = doc[0]
print(f"Word: {first_token.text}")
print(f"Vector (First 10 values): {first_token.vector[:10]}")
print(f"Vector Shape: {first_token.vector.shape}")
```

Task 5 –Text Classification / Prediction

Through this task, we will record and test multiple models for classification of Amazon reviews for sentiment analysis, and we will use Multinomial Naive Bayes and LSTM because they balance between high statistical processing and understanding of context.

Multinomial Naive Bayes (MNB)

Naive Bayes is considered the most efficient and suitable option for the problem of classifying traditional texts dealing with Amazon reviews; after being converted to TF-IDF, it produces thousands of features. This model is appropriate with big arrays and deals with Bayes Theorem with assumes independence words that's make ideal for identifying powerful keywords such as "Excellent" or "Bad" that immediately establish polarity. This model can be used as a starting point to determine the minimum level of accuracy that can be achieved before consuming large resources in complex models. These specifications are compatible with Amazon reviews.

Training MNB Model	Training depends on Supervised Learning technique by learning models linking between the content of the review text and the customer's feelings (positive or negative). Processing training will be in 3 steps:
<pre># Data segmentation (80% training, 20% testing) X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Naive Bayes nb_model = MultinomialNB() nb_model.fit(X_train, y_train) y_pred_nb = nb_model.predict(X_test)</pre>	1. Segmentation Data (80% training, 20% testing). 2. Training. 3. Prediction.

Long Short-Term Memory (LSTM)

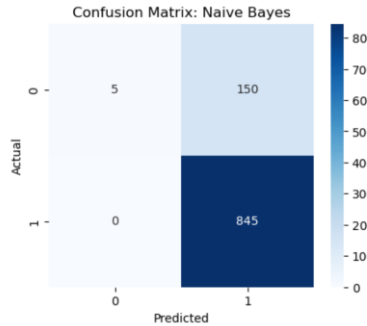
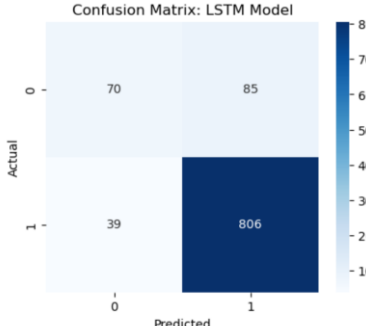
The reason for choosing LSTM is to capture the sequential dependencies in Amazon reviews than traditional models. LSTM maintains a memory of previous words in a sentence, which is essential for understanding context and subtle sentiments in long customer feedback. LSTM is appropriate with Word Embeddings, that's mean it doesn't deal with words as dry symbols, it deal as concepts. If the model learns that "Bad" is negative, it will automatically understand that "Awful" or "Terrible" are also negative, even if it doesn't see them frequently in the training data, because it understands their mathematical position. In addition, this model can deal with distant terms like this iPhone, which I bought for my wife for last birthday is bad.

Training LSTM Model	Training depends on understanding sequence and memory. Processing training will be in 4 steps:
<pre># LSTM Model Preprocessing max_words = 5000 max_len = 100 tokenizer = Tokenizer(num_words=max_words) tokenizer.fit_on_texts(df_sample['Cleaned_Text']) sequences = tokenizer.texts_to_sequences(df_sample['Cleaned_Text']) X_lstm = pad_sequences(sequences, maxlen=max_len) # LSTM data segmentation (80% training, 20% testing) X_train_lstm, X_test_lstm, y_train_lstm, y_test_lstm = train_test_split(X_lstm, y, test_size=0.2, random_state=42) # LSTM Architecture Planning model = Sequential([Embedding(max_words, 128, input_length=max_len), LSTM(64, dropout=0.2, recurrent_dropout=0.2), Dense(1, activation='sigmoid')]) model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy']) # LSTM Model Training print("Starting LSTM Training...") history = model.fit(X_train_lstm, y_train_lstm, epochs=3, batch_size=32, validation_split=0.1) # LSTM Model Evaluation y_pred_lstm = (model.predict(X_test_lstm) > 0.5).astype("int32") plot_cm(y_test_lstm, y_pred_lstm, 'Confusion Matrix: LSTM Model')</pre>	1. Preprocessing: Tokenization, Sequencing and Padding. 2. Architecture Planning: Embedding, LSTM Layer and Dense Layer. 3. Training: Compilation and Fitting. 4. Evaluation: Prediction and Confusion Matrix.

Task 6 – Evaluation, Inferences, Recommendation and Reflection

Evaluation

The evaluation of 2 models depends on group of scientific techniques like Confusion Matrix, Accuracy, Precision, Recall and F1-score based on the attached table:

	Multinomial Naive Bayes (MNB)		Long Short-Term Memory (LSTM)																																																													
Confusion Matrix	 Screenshot 7		 Screenshot 8																																																													
(TN)	5		70																																																													
(FP)	150		85																																																													
(FN)	0		39																																																													
(TP)	845		806																																																													
Accuracy:	MNB Model Accuracy: 85.00%		LSTM Model Accuracy: 86.80%																																																													
	Screenshot 9		Screenshot 10																																																													
	85%		87%																																																													
Reports	MNB Detailed Performance Report <table> <thead> <tr> <th></th><th>precision</th><th>recall</th><th>f1-score</th><th>support</th></tr> </thead> <tbody> <tr> <td>0</td><td>1.00</td><td>0.03</td><td>0.06</td><td>155</td></tr> <tr> <td>1</td><td>0.85</td><td>1.00</td><td>0.92</td><td>845</td></tr> <tr> <td>accuracy</td><td></td><td></td><td>0.85</td><td>1000</td></tr> <tr> <td>macro avg</td><td>0.92</td><td>0.52</td><td>0.49</td><td>1000</td></tr> <tr> <td>weighted avg</td><td>0.87</td><td>0.85</td><td>0.79</td><td>1000</td></tr> </tbody> </table> Screenshot 11			precision	recall	f1-score	support	0	1.00	0.03	0.06	155	1	0.85	1.00	0.92	845	accuracy			0.85	1000	macro avg	0.92	0.52	0.49	1000	weighted avg	0.87	0.85	0.79	1000	LSTM Classification Report <table> <thead> <tr> <th></th><th>precision</th><th>recall</th><th>f1-score</th><th>support</th></tr> </thead> <tbody> <tr> <td>0</td><td>0.59</td><td>0.47</td><td>0.53</td><td>155</td></tr> <tr> <td>1</td><td>0.91</td><td>0.94</td><td>0.92</td><td>845</td></tr> <tr> <td>accuracy</td><td></td><td></td><td>0.87</td><td>1000</td></tr> <tr> <td>macro avg</td><td>0.75</td><td>0.71</td><td>0.72</td><td>1000</td></tr> <tr> <td>weighted avg</td><td>0.86</td><td>0.87</td><td>0.86</td><td>1000</td></tr> </tbody> </table> Screenshot 12			precision	recall	f1-score	support	0	0.59	0.47	0.53	155	1	0.91	0.94	0.92	845	accuracy			0.87	1000	macro avg	0.75	0.71	0.72	1000	weighted avg	0.86	0.87	0.86	1000
	precision	recall	f1-score	support																																																												
0	1.00	0.03	0.06	155																																																												
1	0.85	1.00	0.92	845																																																												
accuracy			0.85	1000																																																												
macro avg	0.92	0.52	0.49	1000																																																												
weighted avg	0.87	0.85	0.79	1000																																																												
	precision	recall	f1-score	support																																																												
0	0.59	0.47	0.53	155																																																												
1	0.91	0.94	0.92	845																																																												
accuracy			0.87	1000																																																												
macro avg	0.75	0.71	0.72	1000																																																												
weighted avg	0.86	0.87	0.86	1000																																																												
	Multinomial Naive Bayes (MNB)		Long Short-Term Memory (LSTM)																																																													
	Macro Average	Weighted Avg.	Macro Average	Weighted Avg.																																																												
Precision	92%	87%	75%	86%																																																												
Recall	52%	85%	71%	87%																																																												
F1-score	49%	79%	72%	86%																																																												

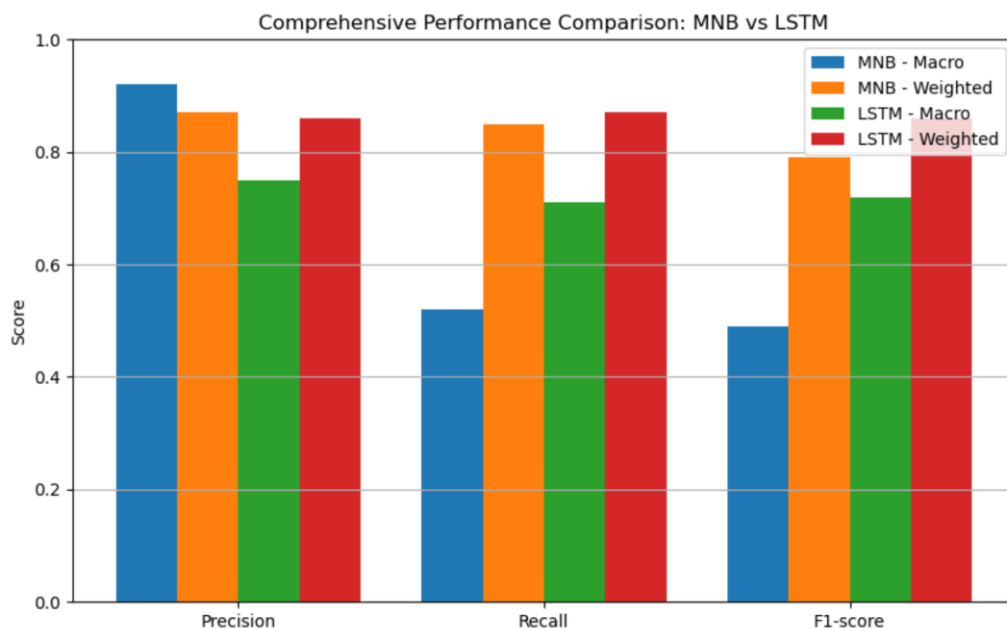
Inferences

After applying the MNB and LSTM models on Amazon reviews for sentiment analysis with classify the review to positive or negative, we inference:

Multinomial Naive Bayes (MNB)	Long Short-Term Memory (LSTM)
The model correctly classified almost all positive instances (TP = 845, FN = 0); however, it showed a clear weakness in identifying negative cases.	The model proves a more balanced classification of both classes, correctly identifying a reasonable number of negative cases (TN = 70), with noticeably fewer errors than MNB and without relying heavily on a single class.

MNB model achieved high macro-averaged precision (92%), indicating accurate predictions when a sentiment class was identified. Also, it showed low macro-averaged recall (52%) and F1-score (49%), suggesting poor detection of one sentiment class, particularly under class imbalance conditions. In Addition, a large gap between macro and weighted averages in MNB indicates a bias toward the majority class.

LSTM displays more balanced performance toward all averages, achieving macro-averaged precision (75%), recall (71%), and F1-score (72%). Strong and fixed overall classification performance depends on weighted average F1-score (86%) reflects. In Addition, the improved performance is attributed to its ability to capture contextual and sequential dependencies in text.



Screenshot 8

According to (Das et al. 2025), MNB display weak performance than other models and bias to positive category in classification of news articles related to the COVID-19 pandemic while LSTM achieved better and more balanced performance.

Overall, the findings indicate that LSTM outperforms MNB in sentiment analysis tasks that require balanced and reliable classification.

Recommendation

- Classify LSTM as approved model as a key tool for sentiment analyses of Amazon reviews according to high ability for understanding context and compound words, and it is exceeded than MNB model.
- Linking the models' results with customer service through automated alert system especially when the result is negative review.
- Future work could focus on training the model on Arabic and additional languages to better support Amazon's diverse global customer base.

Reflections

It was a very interesting project because it's allowed us to clarify our ability to read a dataset, identify a problem, and try to solve it through different models. This invites us to reflect on the machine's ability to create more powerful models for solving more complex problems.

References

Das, Kana, Md Asadullah, Md Murad Hossain, Annita Siddeka Tanni, Shahidul Islam, Masudul Islam, and Mst Sharmin Akter Sumy. 2025. "Deciphering COVID-19 Narratives: A Comparative Study of ML Models (RF, MNB, GB, LR, SVM) and DL Models (CNN, Bi-LSTM) for News Article Classification." *Informatica* 49(14). doi:10.31449/inf.v49i14.6494.

Jurafsky, Dan, and James H. Martin. 2026. *Speech and Language Processing*.

RUMI, ARHAM. 2021. *Amazon Product Reviews*.
<https://www.kaggle.com/datasets/arhamrumi/amazon-product-reviews>.